

SISTEMAS OPERATIVOS

08011442

Germán Sánchez Torres, I.S., M.Sc., Ph.D.

Profesor, Facultad de Ingeniería · Programa de Ingeniería de Sistemas

Universidad del Magdalena, Santa Marta

Teléfono: +57 605 421 4079 ext. 1138 · 301 683 6593

Edificio Docente, cubículo 3D401

Correo: gsanchez@unimagdalena.edu.co · sanchez.gt@gmail.com



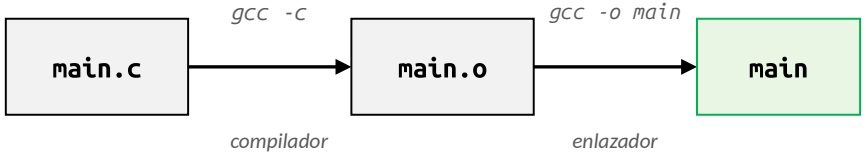
Programación en lenguaje C
Repaso para el curso

Contenido

1. El compilador y el enlazador
2. Escribir y ejecutar un programa; errores de compilación
3. Sintaxis básica: "Hola mundo"
4. Etapas del compilador
5. Variables
6. Funciones y recursión
7. Operadores de comparación y matemáticos
8. Operadores de asignación
9. Bucles: while, do-while y for
10. Punteros
11. Estructuras (struct)
12. Asignación dinámica de memoria

El compilador y el enlazador

- El código fuente (`.c`) se traduce a código objeto (`.o`) y luego se **enlaza** con las bibliotecas para producir un ejecutable.
- El **compilador** (`gcc -c`) traduce cada archivo fuente por separado.
- El **enlazador** (`ld`, invocado por `gcc`) combina los objetos y resuelve referencias a funciones de otros archivos o bibliotecas (como `printf` en la biblioteca estándar).




```
$ gcc -c main.c        # solo compila -> main.o
$ gcc -o main main.o   # solo enlaza -> main
$ gcc -o main main.c   # hace ambos pasos
```

- Un programa grande se compila por módulos: cada `.c` genera su `.o`; el enlazador los une al final.
- El error "undefined reference" ocurre en el enlace, no en la compilación: la función existe en algún lado, pero no se enlazó.

Escribir y ejecutar un programa

1. Escriba el código fuente en un editor de texto y guárdelo con extensión `.c`, por ejemplo `hola.c`.

2. Compile con `gcc` (a la derecha) para obtener el ejecutable `hola`.

3. Si hay errores o advertencias, edite el archivo, corríjalos y vuelva a compilar.

4. Ejecute y observe el resultado: `./hola`.

hola.c

```c
#include <stdio.h>
/* El programa en C mas simple */
int main(void) {
  printf("Hello World\n");
  return 0;
}
```

Compilación y ejecución

```
$ gcc -Wall -Wextra hola.c -o hola
$ ./hola
Hello World
```

El proceso es iterativo: **editar → compilar → corregir → ejecutar**, hasta obtener el resultado esperado.

Leer los mensajes de error del compilador

hola_error.c (con errores a propósito)

```
1 #include <stdio.h>
2 /* Con errores a proposito */
3 int main(void) {
4   int x
5   x = 5;
6   printf("x = %d\n", y);
7   return;
8 }
```

- Línea 4: falta `;` tras `int x` (gcc lo señala en la línea 5).
- Línea 6: `y` no está declarada.
- Línea 7: `main` debe devolver un `int` (advertencia).
- Corrija el primer error y recompile: uno solo puede provocar varios mensajes.

```
$ gcc -Wall -Wextra hola_error.c -o hola_error
hola_error.c: In function 'main':
hola_error.c:5:3: error: expected '=', ',', ';', 'asm' or '__attribute__' before 'x'
hola_error.c:5:3: error: 'x' undeclared (first use in this function)
hola_error.c:5:3: note: each undeclared identifier is reported only once for each function it appears in
hola_error.c:6:22: error: 'y' undeclared (first use in this function)
hola_error.c:7:3: warning: 'return' with no value, in function returning non-void [-Wreturn-type]
hola_error.c:3:5: note: declared here
```

Salida real de gcc 13 (con LANG=C). Formato de cada mensaje: archivo:línea:columna: tipo: descripción.

Sintaxis de C: "Hola mundo"

```c
#include <stdio.h>
/* El programa en C mas simple */
int main(void) {
  printf("Hello World\n");
  return 0;
}
```

Compilar y ejecutar:

```
$ gcc hola.c -o hola
$ ./hola
Hello World
```

Línea	Significado
1	#include inserta otro archivo. Las cabeceras `.h` declaran lo necesario para usar una biblioteca; <...> indica que se busque en los directorios del sistema.
2	Comentario: el compilador lo ignora.
3	`main` es el punto de entrada: la ejecución empieza ahí. `{ ... }` delimita un bloque de código.
4	`printf` imprime el mensaje; `'\n'` es el salto de línea.
5	Valor de retorno: `0` indica que el programa terminó sin errores.

Etapas del compilador

- La compilación ocurre en dos grandes pasos: **preprocesado** y **compilación** (que incluye el enlace).

1. Preprocesado

- Cualquier línea que empieza con # es una directiva:
  - Se insertan los archivos incluidos (`#include`).
  - Se expanden las macros (`#define`).
  - Se eliminan los comentarios (`/* */` , `//`).
  - Se unen las líneas continuadas (`\`).

2. Compilación y enlace

- El compilador convierte el código preprocesado en código objeto.
- El enlazador crea un único ejecutable a partir de los objetos y las bibliotecas, listo para que la CPU lo ejecute.

El comando `gcc -E archivo.c` muestra el resultado del preprocesado, útil para depurar macros e includes.

Variables

- Una variable nombra un lugar en memoria donde se guarda un valor de un tipo determinado.
- Primero se **define** dando un nombre, un tipo y, opcionalmente, un valor inicial.
- El compilador la ubica en algún lugar de la memoria; su dirección se puede obtener con &.

```
char x;
char y = 'e';
```

Símbolo	Dirección	Valor
x	0x1004	? (sin definir)
y	0x1005	'e' (101)

- El valor inicial de x está sin definir: contiene lo que hubiera antes en esa dirección de memoria.
- ¿Qué nombres son legales? Letras, dígitos y guion bajo, sin empezar por dígito.
- ¿Declarar o definir? Declarar anuncia el tipo (p. ej. en un .h); definir además reserva espacio.

Funciones

- Una función es una secuencia de instrucciones que se ejecuta al llamarla. Recibe **argumentos** y devuelve un **valor**.
- `main()` es una función más; solo es especial porque es la primera que se ejecuta al iniciar el programa.
- Llamar a una función: `printf()` es también una función, definida en una **biblioteca** (una colección de funciones reutilizables) que se puede invocar desde cualquier programa.

```c
#include <stdio.h>

/* tipo de retorno, nombre y parametros */
int suma(int a, int b) {
  return a + b;        /* devuelve un valor */
}

int main(void) {
  int r;
  r = suma(2, 3);      /* llamada con argumentos */
  printf("2 + 3 = %d\n", r);  /* otra funcion */
  return 0;
}
```

```
$ ./funciones
2 + 3 = 5
```

Recursión

Recordatorio: la instrucción if / else

```
/* si la expresion evaluada no es 0 */
if (expresion) {
  /* entonces ejecuta este bloque */
}
else {  /* si no, ejecuta este bloque */
}
```

- Una función puede llamarse a sí misma: cada llamada resuelve un **caso base** o se reduce a un caso más pequeño.
- Trazando `potencia(x, 0)`: retorna 1 de inmediato (caso base).
- Trazando `potencia(x, 1)`: `x * potencia(x, 0)` = `x * 1` = x.
- Este razonamiento (resolver el caso base y suponer que el resto ya funciona) se llama **inducción**.

```
#include <stdio.h>
#include <inttypes.h>

float potencia(float x, uint32_t exp) {
  if (exp == 0) {        /* caso base */
    return 1.0;
  }
  /* caso recursivo */
  return x * potencia(x, exp - 1);
}

int main(void) {
  float p = potencia(10.0, 5);
  printf("p = %f\n", p);
  return 0;
}
```

```
$ ./pow_recursiva
p = 100000.000000
```

Operadores de comparación y matemáticos

Comparación y lógicos

`==`	igual a
`<`	menor que
`<=`	menor o igual
`>`	mayor que
`>=`	mayor o igual
`!=`	distinto de
`&&`	y lógico
`||`	o lógico
`!`	no lógico

Aritméticos

`+`	suma
`-`	resta
`*`	multiplicación
`/`	división
`%`	módulo

Bit a bit

`&`	y bit a bit	`~`	negación bit a bit
`|`	o bit a bit	`<<`	desplaza a la izquierda
`^`	xor bit a bit	`>>`	desplaza a la derecha

Cuidado con la división: si ambos operandos son enteros, el resultado se trunca: `5 / 10` → `0`, pero `5 / 10.0` → `0.5`. La división entera por 0 tiene comportamiento indefinido (en Linux, normalmente aborta el programa con SIGFPE).

No confundir & con &&: `1 & 2` → `0` (AND de bits), `1 && 2` → `1` (verdadero, AND lógico).

```c
#include <stdio.h>

int main(void) {
  printf("5 / 10   = %d\n", 5 / 10);     /* 0   */
  printf("5 / 10.0 = %.1f\n", 5 / 10.0); /* 0.5 */
  printf("7 %% 3    = %d\n", 7 % 3);     /* 1   */
  printf("1 & 2    = %d\n", 1 & 2);      /* 0   */
  printf("1 && 2   = %d\n", 1 && 2);     /* 1   */
  printf("5 << 1   = %d\n", 5 << 1);     /* 10  */
  return 0;
}
```

Operadores de asignación

```
x = y    asigna y a x
x++      posincremento de x
++x      preincremento de x
x--      posdecremento de x
--x      predecremento de x
```

Diferencia entre ++x y x++:

```c
#include <stdio.h>

int main(void) {
  int x = 5, y;
  y = ++x;      /* pre:  x == 6, y == 6 */
  printf("y = ++x -> x = %d, y = %d\n", x, y);
  x = 5;
  y = x++;      /* post: x == 6, y == 5 */
  printf("y = x++ -> x = %d, y = %d\n", x, y);
  return 0;
}
```

```
$ ./incremento
y = ++x -> x = 6, y = 6
y = x++ -> x = 6, y = 5
```

```
x += y   asigna (x + y) a x
x -= y   asigna (x - y) a x
x *= y   asigna (x * y) a x
x /= y   asigna (x / y) a x
x %= y   asigna (x % y) a x
```

¡No confundir = con ==! Con `-Wall` el compilador avisa "suggest parentheses" cuando detecta este error.

```c
#include <stdio.h>

int main(void) {
  int x = 5;
  if (x == 6) {      /* comparacion: falso */
    printf("x vale 6\n");
  }
  if (x = 6) {       /* asignacion: verdadero */
    printf("x ahora vale %d\n", x);
  }
  return 0;
}
```

Bucles: while y do-while

```
loop:
  if (condicion) {
    instrucciones;
    goto loop;
  }
```

→

```
while (condicion) {
  instrucciones;
}
```

- El bucle `while` es la forma estructurada del salto con `goto`: repite mientras la condición sea verdadera.

do-while: evalúa la condición al final (se ejecuta al menos una vez):

```
do {
  instrucciones;
} while (condicion);
```

Solución con while:

```
#include <stdio.h>

float potencia(float x, unsigned int exp) {
  unsigned int i = 0;
  float resultado = 1.0;
  while (i < exp) {
    resultado = resultado * x;
    i++;
  }
  return resultado;
}

int main(void) {
  float p = potencia(10.0, 5);
  printf("p = %f\n", p);
  return 0;
}
```

```
$ ./bucles
p = 100000.000000
```

El bucle *for*

```
for (inicio; condición; incremento) {
  instrucciones;
}
```

- Empaqueta en una sola línea lo que `while` expresa en varias: inicialización, condición e incremento.
- Se ejecuta así: inicio una vez; luego, mientras la condición sea verdadera, las instrucciones y el incremento, en ese orden.

while	for

`i = 0;`
`while (i < exp) {`
`  ...`
`  i++;`
`}`
	`for (i = 0; i < exp; i++) {`
	`  ...`
	`}`

```
#include <stdio.h>

float potencia(float x, unsigned int exp) {
  float resultado = 1.0;
  unsigned int i;
  for (i = 0; i < exp; i++) {
    resultado = resultado * x;
  }
  return resultado;
}

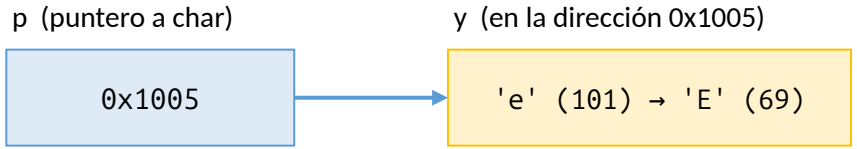
int main(void) {
  float p = potencia(10.0, 5);
  printf("p = %f\n", p);
  return 0;
}
```

```
$ ./for
p = 100000.000000
```

Punteros

- & ("dirección de"): operador de referencia.
- * ("contenido de"): operador de desreferencia.
- Un puntero es una variable cuyo valor es la **dirección** de otra variable; en el ejemplo, p es un "puntero a char".

Dentro de f(&y):



**p = *p - 32 modifica y a través de su dirección.*

```c
#include <stdio.h>
/* &: direccion de   *: contenido de */
void f(char *p) {
  *p = *p - 32;     /* minuscula -> mayuscula */
}

int main(void) {
  char y = 'e';     /* y vale 101 */
  printf("antes: y = %c (%d)\n", y, y);
  f(&y);            /* se pasa la direccion */
  printf("despues: y = %c (%d)\n", y, y);
  return 0;
}
```

```
$ ./punteros
antes: y = e (101)
despues: y = E (69)
```

- Los punteros se usan además para pasar objetos grandes sin copiarlos, acceder a memoria reservada dinámicamente y referirse a funciones.

Estructuras (*struct*)

```c
#include <stdio.h>
/* declaracion de la estructura */
struct my_struct {
  int counter;
  float average;
};

int main(void) {
  struct my_struct x;      /* instancia */
  int sum = 15;
  x.counter = 1;
  x.average = sum / (float) x.counter;
  struct my_struct *ptr = &x;  /* puntero */
  ptr->counter = 2;
  (*ptr).counter = 3;      /* = ptr->counter */
  printf("counter = %d, average = %.2f\n",
         x.counter, x.average);
  return 0;
}
```

- `struct` agrupa varios campos bajo un mismo nombre de tipo.
- Se accede a los campos con `.` sobre la variable, o con `->` sobre un puntero a la estructura.
- `ptr->campo` es exactamente lo mismo que `(*ptr).campo`: ambas formas desreferencian el puntero y acceden al campo.

```
$ ./struct
counter = 3, average = 15.00
```

Asignación dinámica de memoria

- Las variables locales se reservan automáticamente al declararlas: van a la **pila**.
- Si el tamaño depende de una entrada del usuario, en tiempo de ejecución, hace falta **asignación dinámica**: el montículo (*heap*).
- `calloc(n, tam)` reserva *n* elementos de tamaño *tam* en cero; `malloc(bytes)` reserva sin inicializar. Ambas devuelven `NULL` si fallan.
- Toda memoria pedida con `malloc`/`calloc` debe liberarse con `free`.

```
$ ./malloc
v[0] = 0
v[1] = 1
v[2] = 4
v[3] = 9
v[4] = 16
```

```c
#include <stdio.h>
#include <stdlib.h>

int *alloc_ints(size_t n) {
  int *v = calloc(n, sizeof(int));  /* en cero */
  if (v == NULL)
    fprintf(stderr, "sin memoria para %zu\n", n);
  return v;
}

int main(void) {
  size_t n = 5;
  int *v = alloc_ints(n);
  if (v == NULL) return 1;
  for (size_t i = 0; i < n; i++) v[i] = i * i;
  for (size_t i = 0; i < n; i++)
    printf("v[%zu] = %d\n", i, v[i]);
  free(v);                    /* liberar */
  return 0;
}
```

Resumen

- El compilador traduce `.c` a código objeto; el enlazador combina objetos y bibliotecas en un ejecutable.
- Toda variable tiene un tipo, un nombre y, opcionalmente, un valor inicial; el compilador le asigna una dirección de memoria.
- Los bucles `while`, `do-while` y `for` expresan la misma idea con distinta sintaxis.
- Un puntero guarda la dirección de otra variable (&) y se desreferencia con *; es la base de la memoria compartida y de pasar datos por referencia.
- `struct` agrupa campos; `malloc/calloc` y `free` gestionan memoria en tiempo de ejecución.

Referencias y lecturas

- Kernighan, B. W. y Ritchie, D. M. *The C Programming Language*, 2.ª ed. Prentice Hall, 1988.
- Harbison, S. P. y Steele, G. L. *C: A Reference Manual*, 5.ª ed. Prentice Hall, 2002.
- Página de manual: `man gcc`.
- Programas de esta presentación: compilan con `gcc -Wall -Wextra` (`asignacion.c` advierte a propósito). Más ejemplos de C (`assert`, uniones, alineación de estructuras, pragmas) en `example_codes/01-c/` del repositorio del curso.